

제공되는 교안외. pdf3 이후에 작업

Part1. 브랜치자료 push

[Step1: 연결된원격저장소 확인] `git remote -v`

명령어	역할 / 설명	예시
<code>git remote</code>	원격 저장소 이름 목록 보기	<code>git remote</code>
<code>git remote -v</code>	fetch / push URL 모두 보기	<code>git remote -v</code>
<code>git remote add <name> <url></code>	새로운 원격 저장소 추가	<code>git remote add origin https://github.com/pjm9889/ai.git</code>
<code>git remote remove <name></code>	원격 저장소 삭제	<code>git remote remove origin</code>
<code>git remote rename <old> <new></code>	원격 저장소 이름 변경	<code>git remote rename origin github</code>
<code>git remote set-url <name> <url></code>	원격 저장소 URL 변경	<code>git remote set-url origin https://github.com/pjm9889/new.git</code>
<code>git remote show <name></code>	원격 저장소 상세정보(브랜치, HEAD 등)	<code>git remote show origin</code>

[Step2: 원격저장소의 브랜치확인법]
`git ls-remote --heads origin`

목적 / 상황	명령어	특징	예시 출력
원격 저장소에 실제 존재하는 브랜치를 정확하게 확인	<code>git ls-remote --heads origin</code>	★ GitHub(원격)의 브랜치를 가장 정확하게 보여줌	<code>refs/heads/main, refs/heads/apple</code>
원격 브랜치 목록 보기 (origin/브랜치 형태)	<code>git branch -r</code>	로컬에서 tracking 중인 원격 브랜치 포함	<code>origin/main, origin/apple</code>
원격 저장소 전체 정보 확인 (브랜치 + HEAD)	<code>git remote show origin</code>	HEAD 브랜치, 원격 브랜치, tracking 여부까지 표시	<code>Remote branches: main tracked, apple tracked</code>

- 추가조사: 원격브랜치 삭제, 원격브랜치가져오기등

[Step3: push할 브랜치로 이동] `git switch apple`

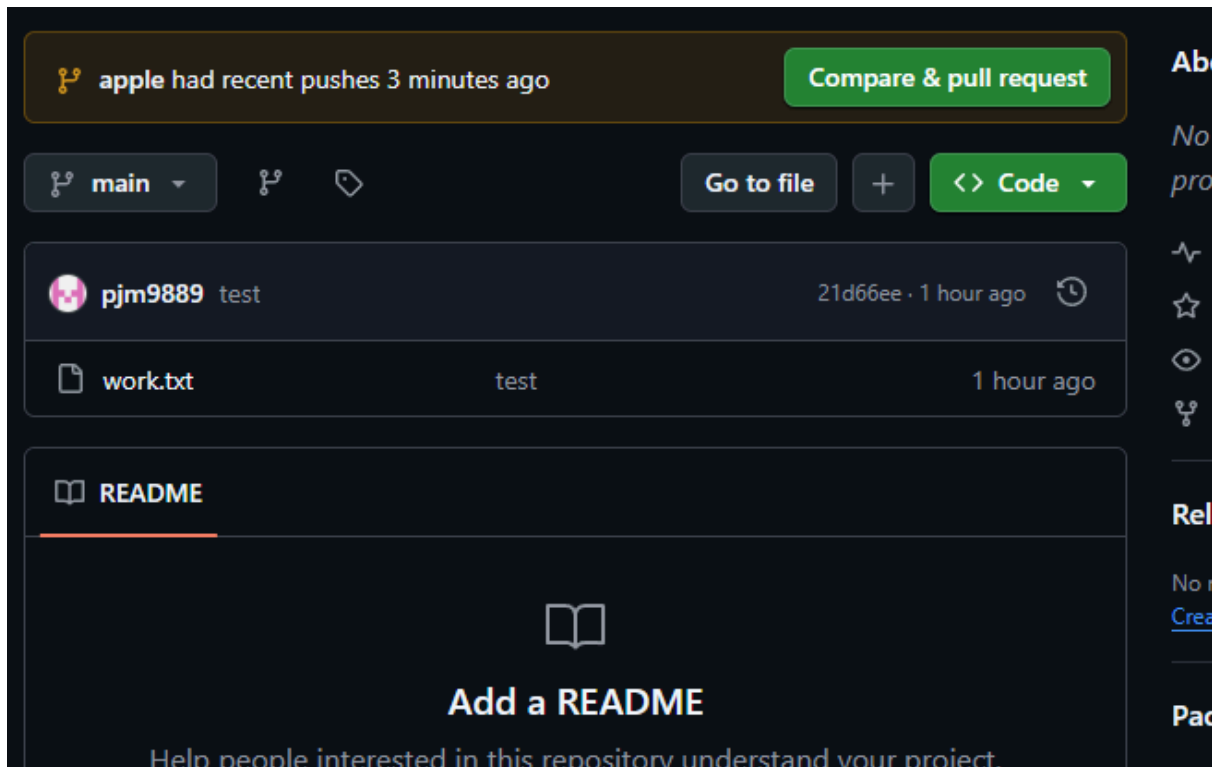
[Step4: git에 push] git push -u origin apple

→ 후 `git ls-remote --heads origin` 로 apple 브랜치 있는지 확인

- push 옵션

옵션	예시	의미 / 언제 씬
-u (또는 --set-upstream)	<code>git push -u origin apple</code>	로컬 apple 브랜치와 원격 apple 브랜치를 연결 (Tracking). 이후엔 <code>git push</code> , <code>git pull</code> 만 써도 자동 연결됨.
-f (또는 --force)	<code>git push -f origin apple</code>	강제 push. 원격 히스토리 덮어쓰기. 위험하지만 히스토리 재작성 후 많이 사용.
--force-with-lease	<code>git push --force-with-lease</code>	덮어쓰긴 하지만, 원격이 내가 예상한 상태일 때만 덮어쓰기 → -f보다 안전.
--all	<code>git push --all origin</code>	로컬의 모든 브랜치 한 번에 push
--tags	<code>git push --tags</code>	로컬에 만든 모든 태그 push
--delete	<code>git push origin --delete apple</code>	원격 apple 브랜치 삭제
--dry-run	<code>git push --dry-run</code>	실제 push는 안 하고 "push 되면 어떻게 될지" 미리 보여줌.
--no-verify	<code>git push --no-verify</code>	pre-push hook 무시하고 push. Hook 에러 있을 때 강제 실행.
--mirror	<code>git push --mirror</code>	브랜치/태그/모든 refs를 원격에 정확히 동일하게 복사. 위험, 거의 안 씬.

[Step5] github에서 병합



Part2. git

.git 폴더는 Git이 프로젝트를 관리하기 위해 사용하는 **숨겨진 메타데이터 디렉토리**로, 이 안에 Git의 모든 이력, 설정, 브랜치 정보 등이 저장됩니다.

- **.git** 폴더는 **절대로 직접 수정하지 마세요!** Git 내부 구조를 잘 알지 못하면 저장소가 손상될 수 있습니다.
- 터미널에서 **.git** 폴더 보기 : `ls -al .git`

```
userbu@DESKTOP-96N9HJ3:~/manual-2$ ls -al .git
total 40
drwxr-xr-x 7 userbu userbu 4096 Dec  4 00:46 .
drwxr-xr-x 3 userbu userbu 4096 Dec  4 00:46 ..
-rw-r--r-- 1 userbu userbu   23 Dec  4 00:46 HEAD
drwxr-xr-x 2 userbu userbu 4096 Dec  4 00:46 branches
-rw-r--r-- 1 userbu userbu   92 Dec  4 00:46 config
-rw-r--r-- 1 userbu userbu   73 Dec  4 00:46 description
drwxr-xr-x 2 userbu userbu 4096 Dec  4 00:46 hooks
drwxr-xr-x 2 userbu userbu 4096 Dec  4 00:46 info
drwxr-xr-x 4 userbu userbu 4096 Dec  4 00:46 objects
drwxr-xr-x 4 userbu userbu 4096 Dec  4 00:46 refs
```

항목	설명
HEAD	현재 체크아웃된 브랜치를 가리킴 (<code>ref: refs/heads/main</code> 등)
config	이 저장소의 Git 설정 정보 (사용자 정보, 원격 저장소 등)
description	Git 저장소 설명 (주로 bare repo에서 사용)
hooks/	커밋 전/후 실행되는 스크립트들이 들어있는 폴더
info/	예외 처리 목록 (<code>exclude</code> 등) 포함
objects/	커밋, 트리, 블롭 등의 실제 Git 객체 데이터
refs/	브랜치와 태그에 대한 참조 정보
logs/	브랜치 이동이나 커밋 히스토리 등의 변경 로그

1 Head

HEAD : Git이 현재 어느 브랜치를 참조 중인지 기록하는 핵심 파일, 지금 어떤 브랜치를 기준으로 작업하고 있는지 확인할 수 있음

[명령] `git branch -m main`

[명령] `userbu@DESKTOP-96N9HJ3:~/manual-2$ cat .git/HEAD`

[출력정보] `ref: refs/heads/main`

```
echo "test" >> aaa.txt
git add .
git commit -m ".git head확인"
[main (root-commit) e68c2ac] .git head확인
1 file changed, 1 insertion(+)
create mode 100644 aaa.txt
```

다음폴더로 이동한뒤 cat main 하여서 정보확인

```
.git/
├── HEAD          ← 현재 가리키는 브랜치 정보 (지금은 main)
└── refs/
    ├── heads/
        └── main  ← 실제 커밋 해시가 저장되는 위치 / 마지막 commit 해시값만 있
```

- `git log`로 보이는 SHA-1 해시값
 - [로그결과예시] `commit a5c80e573cb47a84fd595e6945a8fdbdb3e2005b`

- 해시값: a5c80e573cb47a84fd595e6945a8fdbdb3e2005b
- 객체로 저장됨:
git/objects/a5/c80e573cb47a84fd595e6945a8fdbdb3e2005b
 - 앞 2글자가 디렉토리명, 나머지가 파일명

2 object: 과거 커밋기록 보유

Git의 모든 커밋, 파일, 디렉토리(트리), 태그가 압축된 형태로 저장돼 있음

```
userbu@DESKTOP-96N9HJ3:~/manual-2/.git/objects$ ls -l
total 32
drwxr-xr-x 2 userbu userbu 4096 Dec  4 01:08 4e
drwxr-xr-x 2 userbu userbu 4096 Dec  4 00:59 9d
drwxr-xr-x 2 userbu userbu 4096 Dec  4 01:08 a5
```

항목	설명
4e, 9d, a5	실제 Git 객체들이 저장된 디렉토리 (이름 앞 두 글자)
info/	Git 압축 관련 메타데이터, Git 객체 캐시 등 메타 정보
pack/	Git 객체들을 하나의 압축 파일로 묶어 관리하는 영역 (성능 최적 화용)

• 객체정보

타입	설명
blob(B inary L arge O bject)	실제 파일 내용
tree	디렉토리 구조 (파일 목록 + 권한 등)
commit	커밋 정보 (작성자, 메시지, 부모 커밋 등)
tag	태그 정보 (선택사항)

```
echo "hello" > aaa.txt
git add aaa.txt
git commit -m "add aaa"
하면
```

Git 내부에서 일어나는 일

- Git은 "hello\n" 이라는 **파일 내용을 blob 객체로 저장**
- 이 내용은 SHA-1 해시로 변환되어 `.git/objects/` 에 저장됨
- 트리 객체는 이 blob의 해시값을 참조하며, 파일 이름(`aaa.txt`)과 연결
- `git ls-files --stage :` 로 확
- 특정 객체 정보 보기

```
userbu@DESKTOP-96N9HJ3:~/manual-2/.git/objects$ git cat-file -p a5c80
e573cb47a84fd595e6945a8fdbdb3e2005b
tree 4e974e9b951ad825a80424f42423749df6667a00
parent e68c2acc8d5302c7cdd720ad26c20b1371dce0ec
author 부성순 <bakpak@empas.com> 1764778137 +0900
committer 부성순 <bakpak@empas.com> 1764778137 +0900
```

항목	의미
tree	이 커밋의 파일 상태를 담고 있는 트리 객체 해시
parent	이전 커밋의 해시 (연결된 히스토리)
author	커밋을 만든 사람 정보와 시간
committer	커밋을 실제로 Git에 저장한 사람 (보통 같음)

- tree 객체: 당시의 디렉토리 구조(어떤 파일이 있었는지, 그 파일들은 어떤 해시를 가지는지)를 저장함
- **[명령어로확인하기]** `git cat-file -p`
4e974e9b951ad825a80424f42423749df6667a00

Part3. .gitconfig

.gitconfig = Git 전체 설정을 저장하는 파일 또는 설정 체계

Git에서 사용자 이름 / 이메일 / 에디터 / 병합 방식 / 색상 출력 / 별칭(alias) 등을 저장하는 설정 파일이 바로 **.gitconfig**입니다.

| **.gitconfig** = Git이 동작하는 방식을 정해 놓은 설정 모음

- **.gitconfig의 세 가지 레벨**

- 우선순위: local > global > system
- 이름, 이메일, alias 등 Git 전체 동작 방식 지정

레벨	파일 위치	적용 범위	설명
system	/etc/gitconfig	시스템 전체	해당 PC 모든 사용자에게 적용되는 설정
global	~/.gitconfig	현재 사용자 전체	내가 로그인한 계정에게만 적용
local	.git/config (리포 안)	현재 git 프로젝트만	가장 우선 적용됨

- **설정 확인하는 명령어**

```
git config --list
```

설정	의미
user.email / user.name	Git에서 커밋할 때 사용하는 정보
credential.helper=store	비밀번호를 평문으로 저장(자동 로그인됨)
core.repositoryformatversion=0	Git 기본 형식 버전
core.filemode=true	파일 실행권한 변경 여부 감지
core.bare=false	일반 리포지토리(작업 디렉토리 있음)

설정	의미
core.logallrefupdates=true	모든 ref 업데이트 로깅
remote.origin.url=...	GitHub 원격 저장소 주소
remote.origin.fetch=...	원격 브랜치를 가져오는 규칙